Chapitre 0 — Avant tout | Partie A — Écrire des lettres | leçon 0.0 sur 153 | difficulté 1 / 10

0.0. Le programme qui ne fait rien

Le squelette de tout programme : int main(), et la boucle while.

1. Ce qu'il faut comprendre

`int main()` est le point de départ. Quand la console s'allume, c'est là qu'elle arrive, et nulle part ailleurs. Tout ce que fait le programme s'écrit entre ses accolades `{ }`.

`while (true)` veut dire « tant que vrai » : ce qui est entre ses accolades recommence **sans jamais s'arrêter**. C'est la **boucle de jeu**. Un jeu ne se termine pas tout seul : il attend le joueur.

`image()` attend la prochaine image de l'écran. Il y en a **60 par seconde** : la boucle fait donc un tour tous les soixantièmes de seconde.

Ce programme ne montre rien, mais il tourne. Tous les suivants partent de lui : on ajoutera des choses **avant** la boucle (ce qui se fait une fois) et **dedans** (ce qui se fait à chaque image).

📖 Les mots de cette leçon — si tu débutes

- **accolades { }** — Elles entourent un bloc : les lignes qui vont ensemble (le corps d'une fonction, d'une boucle, d'un « if »). Chaque « { » a son « } ».
- **bloc** — Un groupe de lignes entre accolades { }, qui s'exécutent ensemble.
- **booléen (vrai / faux)** — Le résultat d'une condition : vrai (1) ou faux (0). « true » veut dire vrai, « false » faux.
- **boucle** — Des instructions qui se répètent : « while » (tant que…) ou « for » (pour un compteur qui va de… à…).
- **boucle du jeu** — « while (true) { image(); … } » : ce qui se répète 60 fois par seconde, sans fin, pendant tout le jeu.
- **commentaire (//)** — Tout ce qui suit « // » jusqu'au bout de la ligne : une phrase pour l'humain, que la console ignore.
- **condition** — Une question dont la réponse est vrai ou faux : « score > 10 », « bouton(A) ». On la teste avec « if » ou « while ».
- **image()** — Attend l'image suivante. La console en montre 60 par seconde : c'est ce qui règle la vitesse d'un jeu.
- **instruction** — Un ordre donné à la console, terminé par un point-virgule : « texte(1, 1, "OK"); ».
- **main** — « int main() { … } » : là où le programme commence. La console exécute d'abord la première ligne de main.
- **point-virgule (;)** — Il termine une instruction, comme le point termine une phrase.
- **while (tant que)** — Répète un bloc tant qu'une condition est vraie.

🔍 Le programme, ligne par ligne — chaque ligne dite en français

`// Tout ce qui suit deux barres « // » est un COMMENTAIRE :`

Un commentaire : une phrase pour celui qui lit (« Tout ce qui suit deux barres « // » est un COMMENTAIRE : »). La console l'ignore.

`// la console ne le lit pas, il est là seulement pour toi.`	Un commentaire : une phrase pour celui qui lit (« la console ne le lit pas, il est là seulement pour toi. »). La console l'ignore.
`int main() {  // main = « principal » : quand la console s'allume, elle arrive ICI.`	Le début du programme : la console commence ici, par la première ligne qui suit. (Le commentaire le dit : « main = « principal » : quand la console s'allume, elle arrive ICI. ».)
`// Tout le programme s'écrit entre cette accolade ouvrante`	Un commentaire : une phrase pour celui qui lit (« Tout le programme s'écrit entre cette accolade ouvrante »). La console l'ignore.
`// et la dernière accolade fermante.`	Un commentaire : une phrase pour celui qui lit (« et la dernière accolade fermante. »). La console l'ignore.
`while (true) {  // while (true) = « tant que vrai » : ce qui est entre les`	La boucle du jeu : tout ce qui est entre les accolades recommence sans fin, 60 fois par seconde avec image(). (Le commentaire le dit : « while (true) = « tant que vrai » : ce qui est entre les ».)
`// accolades recommence pour TOUJOURS. C'est la boucle de jeu.`	Un commentaire : une phrase pour celui qui lit (« accolades recommence pour TOUJOURS. C'est la boucle de jeu. »). La console l'ignore.
`image();  // Attend la prochaine image de l'écran (60 par seconde).`	Attendre l'image suivante : la console en montre 60 par seconde. C'est ce qui règle la vitesse du jeu. (Le commentaire le dit : « Attend la prochaine image de l'écran (60 par seconde). ».)
`// Le « ; » termine l'instruction, comme un point en fin de phrase.`	Un commentaire : une phrase pour celui qui lit (« Le « ; » termine l'instruction, comme un point en fin de phrase. »). La console l'ignore.
`}  // Fin de la boucle : on repart à image().`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».
`}  // Fin de main. Ce programme ne montre rien, mais il tourne.`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **71 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
  démarrage       28 o  ← la console, à l'allumage  — la mise en route de la console
  main()           7 o  ← démarrage  — ta fonction
  VBlank           8 o  ← la console, à chaque image  — compte les images, 60 fois par seconde
  AttendreImage   28 o  ← image()  — attend l'image suivante
Lettres gravées : aucune
Pas gravé (jamais appelé) : AvancerAirs, AttendreVBlank, AttendreAcces, AttendreFinVBlank, AdresseCase,
EffacerCases, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins,
InstallerTransfert, CopierTuiles, EffacerCarte, Hasard, EffacerFond, EffacerPanneau, LireManette, Tuiles,
DonneesTransfert, Notes
Fonctions du langage appelées :
  image() × 1     3 o sur place, dans main()  + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// Tout ce qui suit deux barres « // » est un COMMENTAIRE :
// la console ne le lit pas, il est là seulement pour toi.
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {        // main = « principal » : quand la console s'allume, elle arrive ICI.
                    // Tout le programme s'écrit entre cette accolade ouvrante
                    // et la dernière accolade fermante.
  while (true) {    // while (true) = « tant que vrai » : ce qui est entre les
                    // accolades recommence pour TOUJOURS. C'est la boucle de jeu.
    image();        // Attend la prochaine image de l'écran (60 par seconde).
                    // Le « ; » termine l'instruction, comme un point en fin de phrase.
  }                 // Fin de la boucle : on repart à image().
}                   // Fin de main. Ce programme ne montre rien, mais il tourne.
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.

la toute première image — la console vient de démarrer — image 1

4. Ce qu'on doit voir

> Rien : l'écran reste vide, mais la console tourne.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ l'écran est allumé
- ✓ et rien n'y est écrit